

Fiorini Carbognin Gianmario – VR506357
Palamari Catalin – VR500735

Architettura degli Elaborati – Elaborato ASM

Anno Accademico 2023-2024

Consegna:

Si sviluppi un software per la pianificazione delle attività di un sistema produttivo

Indice

1 - Descrizione del progetto	3
2 – Organizzazione progetto	4
3 - Main	5
4 - Stampa a video.....	7
5 - Lettura file	9
6 - Visualizzazione menù	13
7 - Algoritmi di ordinamento	17
8 - Stampa algoritmi.....	21
9 - Trasformazione da intero a stringa.....	22
10 - Test effettuati	26

1 - Descrizione del progetto

L'elaborato ci chiedeva di sviluppare un software per la pianificazione delle attività di un sistema produttivo, fino ad un massimo di 10 prodotti, che andranno realizzati uno alla volta nelle 100 unità di tempo successive dette "slot temporali".

Ogni prodotto è caratterizzato da quattro valori interi:

- **Identificativo:** codice identificativo del prodotto da produrre. Può andare da 1 a 127;
- **Durata:** il numero di slot temporali necessari per completare il prodotto. Ogni prodotto può richiedere 1 a 10 slot temporali;
- **Scadenza:** il tempo massimo entro cui il prodotto dovrà essere completato.
- **Priorità:** un valore da 1 a 5, dove 1 indica la priorità minima e 5 la priorità massima.

Il valore di priorità indica anche la penalità che l'azienda dovrà pagare per ogni unità di tempo necessaria a completare il prodotto oltre la scadenza. Per ogni prodotto completato in ritardo l'azienda dovrà pagare una penale pari al valore della priorità del prodotto completato in ritardo, moltiplicato per il numero di unità di tempo di ritardo rispetto alla sua scadenza.

Il software dovrà essere eseguito dalla linea di comando e caricherà gli ordini dal file selezionato. Il file degli ordini avrà tutti i parametri separati da virgola. (es: 4,10,12,4).

Una volta letto il file, il programma mostrerà il menu principale che chiederà all'utente quale algoritmo di pianificazione dovrà usare: '1' per utilizzare l'algoritmo EDF (verranno prodotti i prodotti con scadenza più vicina. In caso di uguaglianza tra scadenze, verrà realizzato il prodotto con maggiore priorità) e '2' utilizzare l'algoritmo HPF (che andrà a produrre i prodotti con priorità maggiore. In caso di priorità uguale, verrà realizzato il prodotto con scadenza più vicina).

Infine, il software dovrà stampare a video:

1. L'ordine dei prodotti e per ognuno di esse dovrà essere stampata una riga con la seguente sintassi: **ID:Inizio** (ID è l'identificativo del prodotto, Inizio è l'unità di tempo in cui inizia la produzione)
2. L'unità di tempo in cui è prevista la conclusione della produzione dell'ultimo prodotto pianificato.
3. La somma di tutte le penalità dovute a ritardi di produzione.

2 – Organizzazione progetto

Ho diviso il progetto in un totale di 8 file:

- **main.s**: file main del progetto. Richiama le funzioni di lettura del file e di visualizzazione del menù. Inoltre gestisce il controllo del passaggio del parametro e il controllo in caso di passaggio di file vuoto o file inesistente
- **print_text.s**: file di stampa a video. Poiché il progetto richiede numerose stampe a video, abbiamo deciso di ottimizzare il tutto andando a creare un'unica funzione per la stampa a video
- **read_file.s**: viene eseguita la lettura del file contenente i prodotti e il salvataggio dei campi in memoria. Gestisce inoltre il controllo di validità del file.
- **view_menu.s**: file dedicato alla stampa a video del menù, della scelta di una voce (1 EDF, 2 HPF, 3 uscita) e di chiamata alle funzioni relative agli algoritmi di ordinamento e alla funzione di stampa dell'output della produzione. Il file gestisce anche l'immissione di input diverso da 1, 2, 3
- **edf.s** e **hpf.s**: sono i due file che ordinano l' "array" in base al tipo di algoritmo di pianificazione richiesto
- **print_algorithms**: file che permette di stampare a video l'ordine dei prodotti, la conclusione della produzione e la penalità. Per la trasformazione da intero a stringa di ciascun campo richiama la funzione `tostring`
- **tostring**: file che converte i valori numerici in stringhe

3 - Main

Etichette:

```
.section .data

msg_error:
    .ascii "Parametro non inserito\n"

errorlen:
    .long . - msg_error

emptyerr:
    .ascii "Il file e' vuoto, inesistente o non presenta prodotti\n"

emptylen:
    .long . - emptyerr

products:
    .long 0,0,0,0,0,0,0,0,0,0
```

msg_error: stringa che viene visualizzata in caso l'utente non abbia inserito alcun parametro

emptyerr: stringa che verrà visualizzata in caso l'utente abbia inserito un file vuoto o un file inesistente

errorlen/emptylen: rappresentano la lunghezza in byte delle rispettive stringhe

products: etichetta che riserva 40 celle di memoria (1 long = 4 celle da 1B) per la memorizzazione dei prodotti

Codice:

```
_start:
    popl %ebx          # rimuovo dallo stack il numero di paramteri passati
    popl %ebx          # rimuovo dallo stack l'indirizzo di memoria in cui e' memorizzato il nome del programma
    popl %ebx          # salvo in ebx l'indirizzo di memoria in cui e' memorizzato il parametro corrispondente al nome del file da leggere
    cmpl $0, %ebx
    je parameter_error

    pushl $products

    call read_file      # chiamo la funzione read_file

    jmp check_array

print:

    call view_menu

# chiamata della system call di uscita (eax -> salvo il codice della syscall (1), ebx -> salvo il codice di uscita (0))
end:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Il file inizia andando a prendere dallo stack l'indirizzo di memoria della cella in cui è salvato il primo carattere del nome del file. Quando su shell andiamo a scrivere il comando **./bin/pianificatore ordini.txt**, su stack vengono salvati rispettivamente: l'indirizzo di memoria del parametro, l'indirizzo di memoria del path dell'eseguibile e il numero di parametri passati, che saranno due (sono considerati parametri sia il nome del file sia il nome dell'eseguibile). Quindi appena iniziamo il programma, lo stack punterà alla cella in cui è salvato il numero dei parametri passati. Togliamo questo valore con un primo pop, eseguiamo un secondo pop per togliere da stack il path dell'eseguibile e infine facciamo un terzo pop per salvare su `eax` l'indirizzo di memoria del primo carattere della stringa. A questo punto viene fatto un controllo. Se `ebx` vale 0, non sono stati passati parametri e quindi il programma salta alla sezione di codice relativa alla gestione di questo errore. Se è diverso da 0 il programma prosegue.

Andiamo a salvare su stack l'indirizzo di memoria della prima cella riservata al salvataggio dei campi dei prodotti e andiamo a chiamare la funzione di lettura del file. Una volta finita la lettura del file e il salvataggio dei prodotti, torneremo nel main dove verrà eseguito un controllo per vedere se l' "array" è vuoto. **jmp check_array** indica il salto alla sezione di codice in cui verrà controllato l'array. Finito il controllo verrà chiamata la funzione per la visualizzazione del menù.

Una volta finita tutta l'esecuzione del programma, si tornerà nuovamente nel main per chiamare la syscall di chiusura del programma

Gestione errori:

```
# ----- ERRORS -----  
  
parameter_error:  
    pushl errorlen  
    pushl $msg_error  
    call print_text  
    addl $8, %esp  
    jmp end  
  
check_array:  
    leal products, %esi  
    xorl %eax, %eax  
    movb (%esi), %al  
    cmpb $0, %al  
    je empty_array  
  
    jmp print  
  
empty_array:  
    pushl emptylen  
    pushl $emptyerr  
    call print_text  
    addl $8, %esp  
    jmp end  
# stampa il messaggio di errore relativo all'inserimento di file vuoto o file privo di prodotti
```

parameter_error: sezione di codice che si accede se non sono stati passati parametri. Passa alla funzione print_text la lunghezza della stringa di errore e l'indirizzo di memoria della stringa di errore salvandoli su stack, dopodiché salta alla sezione in cui si esce dal programma

check_array: controlla se il primo valore dell' "array" è nullo. In caso affermativo vuol dire che l'array è vuoto ed è stato passato un file vuoto o inesistente. Se l'array è vuoto verrà stampata la stringa di errore, altrimenti si tornerà alla sezione di codice relativa alla visualizzazione del menù

empty_array: equivalente alla logica di parameter_error, stampa il messaggio di errore in caso di file vuoto o inesistente

4 - Stampa a video

Per la stampa a video ho deciso di realizzare un'unica funzione che andasse ad eseguire la syscall di stampa ogni volta che fosse chiamata

```
.section .data

.section .text

.global print_text

.type print_text, @function

print_text:
    pushl %ebp
    movl %esp, %ebp
    pushl %eax
    pushl %ebx
    pushl %ecx
    pushl %edx

    xorl %edx, %edx

    movl $4, %eax
    movl $1, %ebx
    movl 8(%ebp), %ecx
    movb 12(%ebp), %dl
    int $0x80

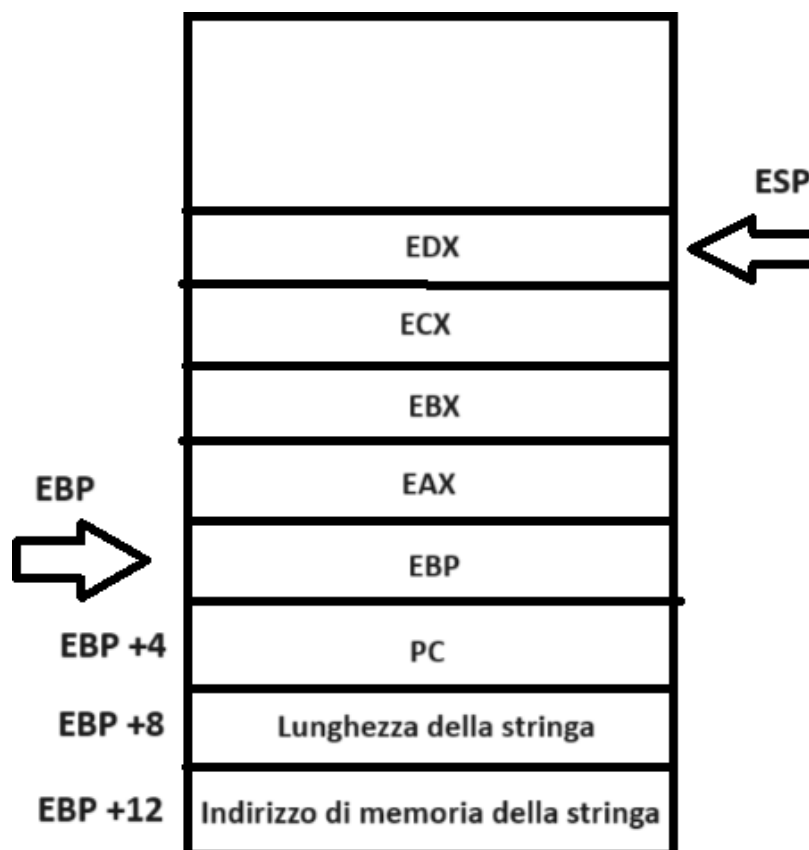
    popl %edx
    popl %ecx
    popl %ebx
    popl %eax

    popl %ebp

    Ret
```


La funzione per prima cosa salva sullo stack il valore di ebp, dato che lo andremo a modificare, e i valori dei 4 registri, in modo che, una volta finita la stampa a video, eax, ebx, ecx ed edx, riprenderanno i precedenti valori

A questo punto viene eseguita la syscall di stampa. In eax viene salvato il valore 4, ovvero il codice della stampa. In ebx viene salvato il codice della stampa su terminale, ovvero 1. In ecx viene salvato l'indirizzo di memoria della stringa da stampare e in edx la lunghezza della stringa da salvare. Sia l'indirizzo della stringa sia la lunghezza sono presi da stack tramite indirizzamento indiretto con spiazzamento tramite ebp. Prima di chiamare print_text occorre salvare su stack la lunghezza e poi l'indirizzo della stringa. Quando verrà chiamata la funzione su stack verrà salvato automaticamente il PC, e quindi per puntare all'indirizzo di memoria della stringa e alla lunghezza, tenendo conto che a inizio funzione salviamo ebp, bisogna far puntare a ebp rispettivamente 8 celle precedenti e 12 celle precedenti



Il disegno precedente ovviamente rappresenta solo la parte di stack relativa alla `print_text`. Notare che ogni cella del disegno sarebbero in realtà 4 celle di memoria, dato che la memoria lavora a BYTE e noi salviamo ogni informazione come 32 bit

5 - Lettura file

Il file `read_file.s` è una funzione che legge il file passato come parametro e salva i valori di ciascun prodotto nell'etichetta **products**, ovvero l'etichetta con 40B di memoria riservati per la memorizzazione dei campi

Etichette:

```
.section .data

invaliderr:
    .ascii "File inserito non valido\n"

invalidlen:
    .long . - invaliderr

fd:
    .long 0

buffer:
    .string ""
```

invaliderr: stringa che verrà visualizzata in caso il file non fosse valido

invalidlen: lunghezza dell'etichetta `invaliderr`

fd: etichetta in cui verrà salvato il file descriptor

buffer: etichetta in cui verrà salvato di volta in volta ciascun carattere letto da file

Codice:

```
read_file:
    pushl %ebp
    movl %esp, %ebp
    movl 8(%ebp), %esi
```

Il file inizia andando a salvare su stack ebp per non perdere il valore di ebp precedente alla chiamata della funzione. Successivamente si fa puntare ebp alla stessa cella di memoria di esp e si usa l'indirizzamento indiretto con spiazzamento per fare puntare a ebp alla cella di memoria in cui è memorizzato l'indirizzo dell'etichetta **products**, e salvando il rispettivo indirizzo in esi. Da qui in poi esi punterà alla prima cella di memoria dell' "array"

```
open_file:
    movl $5, %eax
    movl $0, %ecx
    int $0x80

    cmpl $0, %eax
    jl exit_read

    mov %eax, fd
```

Sezione di codice in cui viene eseguita la syscall di apertura del file. In eax viene salvato il codice relativo alla syscall di apertura file, in ebx ci deve essere l'indirizzo di memoria del nome del file, che è già presente a questo punto del programma (lo abbiamo già salvato dal main prendendolo da stack) e infine in ecx salvo il codice relativo alla modalità di lettura del file, ovvero 0. La syscall salverà su eax il codice del file descriptor. Se è un valore minore di 0, si è verificato un errore in apertura e il programma salterà alla sezione di codice relativa alla chiusura del programma e uscita dalla funzione. Altrimenti, il codice del file descriptor viene salvato in memoria

```
# sezione di codice che esegue la lettura del file carattere per carattere
read_loop:
    movl $3, %eax          # in eax salvo il codice della system call READ
    movl fd, %ebx          # in ebx salvo il valore identificativo del file da cui voglio leggere
    leal buffer, %ecx      # in ecx salvo l'indirizzo di memoria della stringa in cui salvare il valore letto
    movl $1, %edx          # in edx salvo il numero di caratteri che voglio leggere
    int $0x80

    cmpl $0, %eax          # verifico la presenza di eventuali errori oppure verifico che il file non sia arrivato alla fine (EOF)
    jle close_file        # in caso di presenza di errori oppure in caso di arrivo alla fine del file viene eseguito il salto alla sezione di chiusura del file

    movb buffer, %bl       # salvo in bl il carattere appena letto

# sezione di codice dedicata al controllo dei caratteri
check_characters:
    cmpb $44, %bl          # se il carattere appena letto corrisponde al carattere ',' ho terminato di salvare il valore di un campo e quindi...
    je scroll_array        # ... scorro l'array alla cella successiva
    cmpb $13, %bl          # se il carattere appena letto corrisponde al carattere '\n' ho terminato di salvare l'ultimo valore di un prodotto e quindi...
    je scroll_array        # ... scorro l'array alla cella successiva
    cmpb $10, %bl          # se il carattere appena letto corrisponde al valore di nuova riga...
    je read_loop          # ... torno al read_loop

    # se il carattere non e' numerico (apparte ',', e '\n') salto alla sezione di codice relativa alla gestione di file non valido
    cmpb $48, %bl
    jl invalid_file
    cmpb $57, %bl
    jg invalid_file

# sezione di codice dedicata al salvataggio di ciascun campo su vettore
store_values:
    movl (%esi), %eax      # carico in eax il valore della cella di memoria puntata da esi
    movl $10, %edx         # salvo il valore 10 su edx per fare poi la moltiplicazione per la logica -> num = num * 10 + cifra
    subb $48, %bl          # trasformo il codice ascii della cifra nella cifra vera e propria
    mullb %edx             # EAX = EAX * 10
    addl %ebx, %eax        # aggiungo ad eax la cifra salvata in ebx
    movl %eax, (%esi)      # salvo il valore nella cella di memoria puntata da esi
    jmp read_loop
```

A questo punto del programma inizia il ciclo di lettura di carattere per carattere dal file, controllo del carattere e salvataggio di ciascun campo su “array”

read_loop: viene eseguita la syscall di lettura. In eax viene salvato il codice relativo alla syscall di apertura, in ebx ci deve essere il codice relativo del file descriptor, che è stato precedentemente salvato nell’etichetta fd. In ecx ci deve essere l’indirizzo di memoria in cui verrà salvato il carattere letto, ovvero l’etichetta **buffer**. In edx salvo il numero di caratteri che andrò a leggere, cioè 1.

In eax verrà salvato un certo valore. Se esso è minore o uguale di 0, vorrà dire che si è verificato un errore (se < 0) oppure siamo arrivati alla fine del file (= 0). In questo caso saltiamo alla sezione di codice relativa alla chiusura del file e uscita dalla funzione. Se tutto procede

normalmente, viene salvato su bl il carattere memorizzato nel buffer **check_characters:** viene eseguito il controllo del valore del carattere. Se bl, che contiene il valore del carattere, è uguale al carattere ‘,’ o ‘\n’, vuol dire che abbiamo finito di leggere un intero campo e quindi è necessario saltare alla sezione di codice che incrementa esi per puntare alla cella dell’ “array” successiva. Se il carattere è ‘\r’, ovvero il carattere di carriage return, non proseguo l’esecuzione ma vado a leggere un nuovo carattere. Dopodiché controllo che i caratteri dei file siano validi. Se il carattere letto è un valore minore di 48 o maggiore di 57, vuol dire che non è un carattere numerico e quindi il programma salterà alla sezione di gestione dell’errore di invalidità del file.

store_values: la logica di questa sezione è quella di salvare di volta in volta il valore progressivo del campo corrente con la logica del **num = num * 10 + cifra**. Si parte salvando in eax il valore presente nella cella puntata da esi, che inizialmente sarà a 0. Dopodiché si salva in edx il valore 10, si esegue la moltiplicazione con edx (la moltiplicazione prende il valore di eax/ax/al, rispettivamente mull, mulw e mulb, e lo moltiplica per il registro specificato dopo il comando). Eseguita la moltiplicazione occorre sottrarre a bl il valore 48, per trasformare la codifica ascii della cifra nella cifra effettiva. La cifra viene sommata a eax e poi il valore progressivo viene salvato nella cella puntata da esi. Finito questo si va a leggere il carattere successivo.

```

# scorrimento del vettore
scroll_array:
    inc %esi                # faccio puntare esi alla successiva cella di memoria riservata al vettore
    jmp read_loop

close_file:
    movl $6, %eax
    movl %ebx, %ecx
    int $0x80

exit_read:
    popl %ebp                # riassegno ad ebp il precedente valore

    Ret

```

scroll_array: incremento esi per far puntare alla cella successiva in modo da salvare il campo successivo

close_file: eseguo la syscall di chiusura del file. La syscall di chiusura file necessita il relativo codice salvato su eax e il file descriptor in ecx. A questo punto del codice il file descriptor si trova in ebx

exit_read: riassegno a ebp il suo precedente valore ed esco dalla funzione

Gestione errori:

```

# ERRORS
# sezione di codice relativa alla gestione del processo in caso di inserimento di file non valido
invalid_file:
    pushl invalidlen        # salvo su stack la lunghezza della stringa di errore
    pushl $invaliderr       # salvo su stack l'indirizzo di memoria della stringa di errore
    call print_text         # stampo il messaggio di errore
    addl $8, %esp
    jmp error_exit

error_exit:
    movl $6, %eax
    movl %ebx, %ecx
    int $0x80                # chiudo il file

    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80                # esco dal programma

```

sezione di codice che accedo solamente se il file contiene caratteri non validi. Viene stampato l'errore di invalidità del file. Successivamente verrà chiuso il file e verrà chiuso il programma.

6 - Visualizzazione menù

File dedicato alla visualizzazione a video del menù, della lettura del codice della voce selezionata e della chiamata agli algoritmi di ordinamento

Etichette:

```
.section .data

title:
    .ascii "Menu' pianificatore\n"

title_len:
    .long . - title

menu_item1:
    .ascii "1 - Pianificazione EDF (Earliest Deadline First)\n"

item1_len:
    .long . - menu_item1

menu_item2:
    .ascii "2 - Pianificazione HPF (Highest Priority First)\n"

item2_len:
    .long . - menu_item2

menu_item3:
    .ascii "3 - Esci\n"

item3_len:
    .long . - menu_item3

item_selection:
    .ascii "\nSeleziona una voce: "

selection_len:
    .long . - item_selection
```

Queste etichette rappresentano le stringhe che verranno immediatamente stampate una volta entrati nella funzione, con le relative lunghezze

```

input:
    .ascii "00"

input_error:
    .ascii "E' stata inserita una voce non valida. Inserisci una valore tra 1, 2 o 3\n"

inputerr_len:
    .long . - input_error

edf_title:
    .ascii "\nPianificazione EDF:\n"

edf_len:
    .long . - edf_title

hpf_title:
    .ascii "\nPianificazione HPF:\n"

hpf_len:
    .long . - hpf_title

```

input: stringa per la memorizzazione dell'input relativo alla selezione della voce. E' formato da due byte perché l'input è formato dal carattere numerico immesso e dall'andata a capo. Usando solamente un byte veniva stampato più volte la richiesta di selezione della voce

input_error: stringa che viene stampata in caso l'utente immette un valore di input diverso da 1, 2 o 3

edf_title e **hpf_title:** stringhe che verranno stampate se sono stati scelti rispettivamente l'algoritmo edf o hpf

Codice:

```

view_menu:
    # lettura del titolo del menu'
    pushl title_len
    pushl $title
    call print_text
    addl $8, %esp

    # lettura delle 3 voci
    pushl item1_len
    pushl $menu_item1
    call print_text
    addl $8, %esp

    pushl item2_len
    pushl $menu_item2
    call print_text
    addl $8, %esp

    pushl item3_len
    pushl $menu_item3
    call print_text
    addl $8, %esp

```

Questa sezione di codice stampa a video il menù:

```
Menu' pianificatore
1 - Pianificazione EDF (Earliest Deadline First)
2 - Pianificazione HPF (Highest Priority First)
3 - Esci
```

```
select_item:
    pushl selection_len
    pushl $item_selection
    call print_text
    addl $8, %esp

    movl $3, %eax
    movl $1, %ebx
    leal input, %ecx
    movl $2, %edx

    int $0x80

    movb input, %al
    subb $48, %al
    cmpb $1, %al
    jl selection_error
    cmpb $3, %al
    jg selection_error
    cmpb $3, %al
    je close_menu

    cmpb $2, %al
    je hpf_algorithm
```

Questa sezione di codice gestisce la richiesta di selezione di una voce del menù con la relativa gestione dell'input dell'utente. I primi 4 comandi servono a stampare la stringa di richiesta di input dell'utente:

```
Seleziona una voce:
```

I successivi 5 comandi invece chiamano la syscall di lettura dell'input da terminale, che verrà salvato nell'etichetta "input". Il valore di input, che al

momento è un carattere ascii, viene salvato in al. Viene sottratto il valore 48 per trasformare il carattere numerico nel suo rispettivo valore numerico. A questo punto viene eseguito il controllo del valore. Se il valore numerico è un valore minore di 1 o maggiore di 3, è stato dato in input un valore non valido e il programma salta alla sezione di codice di gestione dell'input non valido. Se il valore corrisponde a 3, il programma salta alla sezione di ritorno della funzione. Se il valore è 2 il programma salta alla sezione di codice relativa all'esecuzione dell'algoritmo hpf. Se non viene eseguito alcun salto verrà direttamente eseguito l'algoritmo di pianificazione edf

```
edf_algorithm:
    pushl edf_len
    pushl $edf_title
    call print_text
    addl $8, %esp

    call edf
    call print_algorithm
    jmp select_item
```

I primi quattro comandi permettono di stampare a video la stringa relativa alla pianificazione edf:

Pianificazione EDF:

Successivamente alla stampa vengono chiamate la funzione edf che si occupa dell'ordinamento dell' "array" secondo l'algoritmo edf, e la funzione print_algorithms che esegue la stampa a video nel formato:

```
4:0
12:10
Conclusione: 17
Penalty: 0
```

Tutta questa parte di codice, dalla stampa a video del menù alla stampa dell'algoritmo, viene eseguita finché l'utente non immette il valore 3.

La sezione hpf è identica alla sezione edf, cambia solo la stringa stampata e la chiamata alla funzione hpf.

Gestione errori:

```
selection_error:
    pushl inputerr_len
    pushl $input_error
    call print_text
    addl $8, %esp
    jmp select_item
```

Se l'utente digita come input un valore diverso da 1, 2, 3, verrà stampata la stringa di avviso di immissione errata e verrà nuovamente richiesta l'immissione di un input.

```
Seleziona una voce: 6
E' stata inserita una voce non valida. Inserisci una valore tra 1, 2 o 3
Seleziona una voce:
```

7 - Algoritmi di ordinamento

Sia il file edf.s che il file hpf.s dovranno andare ad ordinare l'array in base al proprio algoritmo. Per fare ciò è stata implementata la logica del for annidato.

Quello che verrà mostrato è il codice del file edf. Il codice del file hpf è simile

Codice:

```
edf:
    pushl %ebp
    movl %esp, %ebp
    movl 12(%ebp), %esi

    xorl %edx, %edx
```

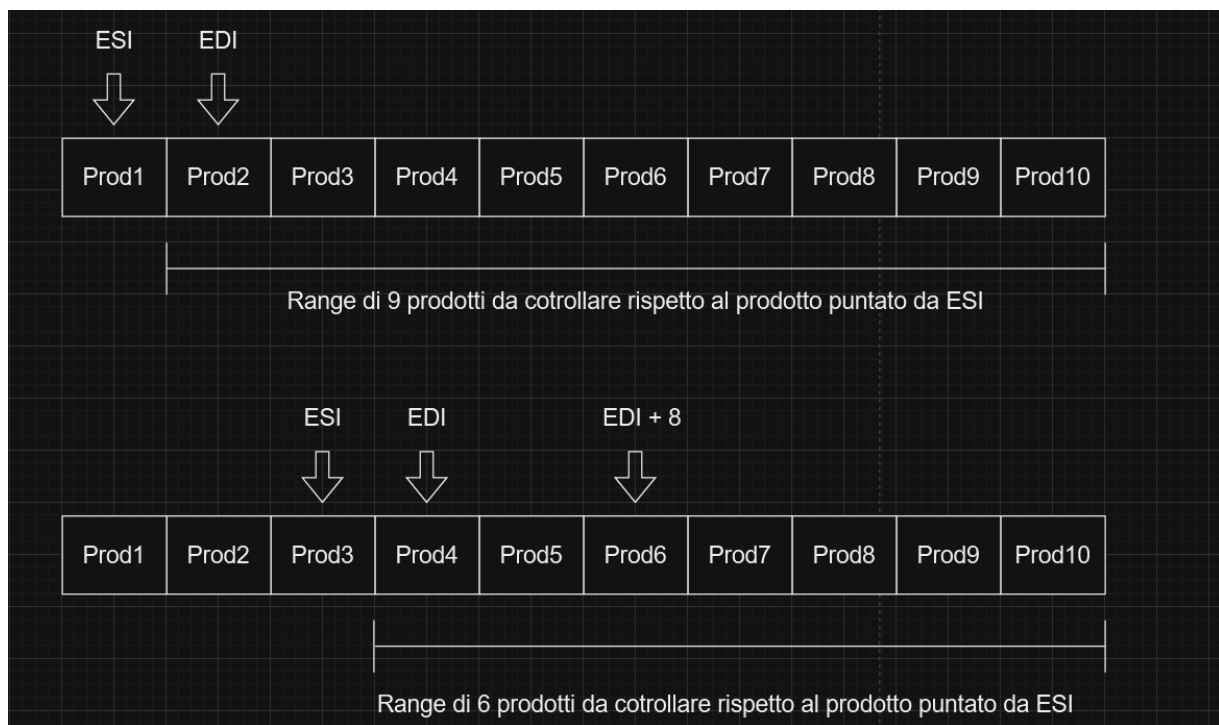
In esi viene salvato l'indirizzo della prima cella di memoria riservata ai valori numerici dei campi dei prodotti, reperito da stack

```
edf_sort:
    movl %esi, %edi
    addl $4, %edi
    movl $9, %ecx
    subl %edx, %ecx

edf_loop:
    xorl %eax, %eax
    movb 2(%esi), %al
    xorl %ebx, %ebx
    movb 2(%edi), %bl
    cmpb $0, %al
    je edf_end
    cmpb $0, %bl
    je edf_loop_continue
    cmpb %al, %bl
    je edf_priority_sort
    cmpb %al, %bl
    jg edf_loop_continue
```

In edi viene salvato l'indirizzo di memoria della prima cella di memoria riservata ai prodotti sommato di 4. In questo modo edi punta alla prima cella di memoria del successivo prodotto, e potrà essere utilizzato per confrontare il prodotto puntato da esi con i prodotti successivi. ecx viene usato come contatore del primo ciclo. edx viene usato per contare di quanti prodotti si è spostato esi (inizialmente a 0 poiché esi punta al primo prodotto). In questo modo, ogni volta che esi passerà al prodotto

successivo, con il comando `subl edx, ecx` è possibile trovare l'esatto numero di prodotti successivi ad `esi` che dovranno essere confrontati. Ad esempio, se nel corso del programma `esi` punta al terzo prodotto, non dovrò eseguire 9 volte il controllo dei prodotti successivi, perché uscirei dalla porzione di memoria riservata ai prodotti, ma dovrò eseguire il controllo 9 - 3 volte



edf_loop: ciclo che confronta il prodotto puntato da `esi` con quelli successivi. In `al` viene salvato il campo scadenza del prodotto puntato da `esi`, in `bl` quello puntato da `edi`. Innanzitutto viene eseguito un controllo su `al` e `bl`: se `al` vale 0, sono i prodotti da confrontare sono finiti e si salterà alla sezione di codice di uscita del programma. Se `bl` vale 0, vuol dire che sono già stati confrontati tutti i prodotti successivi a quello puntato da `esi`, si salterà quindi alla sezione di incremento di `esi` al prodotto successivo: Da notare che queste condizioni si verificano SOLO se il file contiene un numero di prodotti minore di 10. Finiti questi due controlli, si fanno due ulteriori controlli: se `al` e `bl` sono identici, i due prodotti hanno scadenza uguale e si andrà a confrontare le rispettive priorità. Se invece `bl` è più piccolo di `al`, il prodotto puntato da `edi` ha scadenza più vicina e verrà eseguito l'invertimento.

```

edf_deadline_sort:
    movb %bl, 2(%esi)
    movb %al, 2(%edi)

    movb (%esi), %al
    movb (%edi), %bl
    movb %bl, (%esi)
    movb %al, (%edi)

    movb 1(%esi), %al
    movb 1(%edi), %bl
    movb %bl, 1(%esi)
    movb %al, 1(%edi)

    movb 3(%esi), %al
    movb 3(%edi), %bl
    movb %bl, 3(%esi)
    movb %al, 3(%edi)

    jmp edf_loop_continue

```

In questa sezione viene eseguito l'invertimento dei prodotti prendendo come riferimento la scadenza. Vengono scambiati prima le due scadenze e successivamente gli id, le durate e le priorità. La sezione di invertimento secondo la priorità è identica, cambia solo il tipo di controllo.

Tutti i controlli vengono eseguiti finché non sono stati confrontati tutti i prodotti, cioè fin quando non si conclude la logica dei for annidati

8 - Stampa algoritmi

Il file `print_algorithms` si occupa della stampa della produzione dei prodotti secondo l'algoritmo scelto

Etichette:

```
.section .data

numstr:
    .ascii "0000"

conclusionstr:
    .ascii "Conclusione: "

conclusionlen:
    .long . - conclusionstr

penaltystr:
    .ascii "Penalty: "

penaltylen:
    .long . - penaltystr

conclusion:
    .byte 0

penalty:
    .byte 0
```

numstr: stringa relativa al valore di un campo numerico. Necessario per la stampa a video

conclusion: verrà salvato il valore numerico relativo all'unità temporale di fine produzione

penalty: viene salvato di volta in volta il totale dell'importo che l'azienda deve pagare come penalità

Codice:

```
print_algorithm:
    pushl %ebp
    movl %esp, %ebp
    movl 12(%ebp), %esi

    movl $10, %ecx
```

Salvo in esi il l'indirizzo della prima cella di memoria riservata ai prodotti

```
print_info:
    pushl %ecx

    xorl %eax, %eax

    movb (%esi), %al

    cmpb $0, %al
    je loop_continue

    pushl %eax
    pushl $numstr
    call tostring
    addl $8, %esp

    movb $58, (%edx,%ebx,1)
    incl %edx

    pushl %edx
    pushl $numstr
    call print_text
    addl $8, %esp

    xorl %eax, %eax
    movb conclusion, %al

    pushl %eax
    pushl $numstr
    call tostring
    addl $8, %esp

    movb $10, (%edx,%ebx,1)
    incl %edx

    pushl %edx
    pushl $numstr
    call print_text
    addl $8, %esp
```

Per ogni prodotto viene stampato prima la parte “id:” e poi la parte “inizio\n”. Per prima cosa salvo in eax l’identificativo del prodotto puntato da esi. Controllo che eax non valga 0, altrimenti ho finito di stampare tutti i prodotti (succede SOLO se i prodotti nel file sono minori di 10).

L’identificativo e l’indirizzo di memoria della stringa in cui verrà salvato l’ascii del rispettivo valore numerico vengono salvati su stack e verranno utilizzati dalla funzione tostring, che convertirà il valore numerico dell’id in stringa. Alla stringa viene aggiunto il carattere ‘:’ e il tutto viene stampato a video. Il procedimento viene ripetuto anche per il valore di inizio produzione, che è salvato nell’etichetta conclusion (inizialmente a 0).

```
xorl %eax, %eax
movb 1(%esi), %al
addb %al, conclusion

penalty_calc:
xorl %ebx, %ebx
movb 2(%esi), %bl
movb conclusion, %al

cmpb %bl, %al
jle loop_continue
subl %ebx, %eax
xorl %ebx, %ebx
movb 3(%esi), %bl
mulb %bl
addb %al, penalty
```

A questo punto all’etichetta conclusion viene sommato il valore della durata e la penalità, solamente se scadenza è minore del valore di fine produzione, viene calcolata dalla differenza tra il valore di fine produzione del prodotto corrente (conclusion) e la scadenza (bl), il tutto moltiplicato per la priorità (bl)

Questo processo viene ripetuto finché non sono stati letti tutti i prodotti. Dopodiché viene eseguita la stampa a video della conclusione della produzione e della penalità, che funziona ugualmente al processo descritto sopra.

9 - Conversione intero-stringa

Il file tostring si occupa della conversione di un valore numerico nella rispettiva stringa

Etichette:

```
.section .data  
  
strtmp:  
    .ascii "000"
```

L'unica etichetta presente è la **strtmp**, la stringa in cui verrà stampato il valore numerico invertito

Codice:

```
tostring:  
    pushl %ebp  
    movl %esp, %ebp  
  
    leal strtmp, %edi  
  
    xorl %eax, %eax  
    xorl %ebx, %ebx  
    movl $10, %ebx  
    xorl %ecx, %ecx  
  
    movb 12(%ebp), %al
```

In edi viene salvato l'indirizzo di memoria della stringa temporanea. In ebx viene salvato il valore 10, dato che andremo a dividere il valore numerico da convertire per 10 e salvare il suo resto. In eax viene salvato il valore numerico da convertire

```

start_loop:
    xorl %edx, %edx
    divl %ebx
    addb $48, %dl
    movb %dl, (%ecx,%edi,1)
    inc %ecx
    cmpl $0, %eax
    jne start_loop

    xorl %edx, %edx

    movl 8(%ebp), %ebx

```

Il valore numerico da convertire viene diviso per 10 finché non vale 0. Ad ogni divisione, il resto salvato su dl viene sommato a 48 per renderlo un carattere e viene salvato in edi attraverso l'indirizzamento base + offset. Alla fine di questa sezione nella stringa temporanea ci sarà salvata la stringa corrispondente al valore numerico invertito.

```

reverse:
    xorl %eax, %eax
    movb -1(%ecx,%edi,1), %al
    movb %al, (%edx,%ebx,1)

    inc %edx

    loop reverse

```

Sezione di codice che inverte la stringa del valore numerico convertito e salva il tutto nella stringa salvata su stack nella funzione print_algorithms

10 - Test effettuati

1° test effettuato: normale esecuzione. Sono stati effettuati i test con i file richiesti, ovvero EDF.txt (penalità a 0 con EDF e > 0 con HPF), Both.txt (entrambi gli algoritmi hanno priorità pari a 0) e None.txt (entrambi gli algoritmi hanno penalità > 0)

EDF:

```
Menu' pianificatore
1 - Pianificazione EDF (Earliest Deadline First)
2 - Pianificazione HPF (Highest Priority First)
3 - Esci

Seleziona una voce: 1

Pianificazione EDF:
117:0
34:10
98:15
9:23
105:30
3:35
20:45
52:54
78:60
Conclusione: 70
Penalty: 0

Seleziona una voce: 2

Pianificazione HPF:
117:0
9:10
105:17
98:22
20:30
3:39
78:49
34:59
52:64
Conclusione: 70
Penalty: 75
```

Both:

```
Pianificazione EDF:
4:0
80:8
49:13
108:19
127:28
19:32
31:39
67:44
73:49
Conclusione: 59
Penalty: 0

Seleziona una voce: 2

Pianificazione HPF:
4:0
80:8
49:13
108:19
127:28
19:32
31:39
67:44
73:49
Conclusione: 59
Penalty: 0
```

None:

```
Seleziona una voce: 1
Pianificazione EDF:
8:0
16:8
12:15
32:23
24:28
50:37
88:44
80:52
90:62
100:67
Conclusione: 70
Penalty: 70

Seleziona una voce: 2
Pianificazione HPF:
88:0
80:8
8:18
32:26
24:31
90:40
12:45
100:53
16:56
50:63
Conclusione: 70
Penalty: 227
```

2° test effettuato: esecuzione del programma senza immissione di parametro

```
gianmario@giammissp:/mnt/c/Users/fgiam/Desktop/VR506357_VR500735$ ./bin/pianificatore
Parametro non inserito
gianmario@giammissp:/mnt/c/Users/fgiam/Desktop/VR506357_VR500735$ |
```

3° test effettuato: inserimento di file non validi

```
gianmario@giammissp:/mnt/c/Users/fgiam/Desktop/VR506357_VR500735$ ./bin/pianificatore Ordini/invalid.txt
File inserito non valido
```

File **invalid.txt**: questo file contiene solo stringhe

```
1 Elaborato di architettura
2 Secondo periodo
3 Progetto assembly
4 Pianificatore
5
```

4° test effettuato: inserimento di un file vuoto, di un file contenente solo caratteri ',' e di un file non esistente

```
gianmario@giammissp:/mnt/c/Users/fgiam/Desktop/VR506357_VR500735$ ./bin/pianificatore Ordini/empty.txt
Il file e' vuoto, inesistente o non presenta prodotti
gianmario@giammissp:/mnt/c/Users/fgiam/Desktop/VR506357_VR500735$ ./bin/pianificatore Ordini/noproducts.txt
Il file e' vuoto, inesistente o non presenta prodotti
gianmario@giammissp:/mnt/c/Users/fgiam/Desktop/VR506357_VR500735$ ./bin/pianificatore Ordini/pippo.txt
Il file e' vuoto, inesistente o non presenta prodotti
```

Il file **empty.txt** è vuoto. Il file **pippo.txt** non esiste. Il file **noproducts.txt** è composto da:

```
1  ,,,  
2  ,,,  
3  ,,,  
4  ,,,  
5  ,,,  
6  ,,,  
7  ,,,  
8  ,,,  
9  ,,,  
10 ,,,  
11 ,,,
```

5° test effettuato: inserimento di file con solamente 3 prodotti

```
Menu' pianificatore
1 - Pianificazione EDF (Earliest Deadline First)
2 - Pianificazione HPF (Highest Priority First)
3 - Esci

Seleziona una voce: 1

Pianificazione EDF:
12:0
8:8
16:16
Conclusione: 23
Penalty: 31

Seleziona una voce: 2

Pianificazione HPF:
16:0
8:7
12:15
Conclusione: 23
Penalty: 40
```